

An Introduction to Dynamic Programming

What is Dynamic Programming?

Dynamic Programming (DP) is a method for solving complex problems by breaking them down into simpler subproblems. It is applicable when a problem has:

- **Overlapping subproblems:** The problem can be broken down into subproblems which are reused multiple times.
- **Optimal substructure:** The optimal solution of the problem can be constructed from the optimal solutions of its subproblems.

Let's explore each of these in detail with examples.

1. Overlapping Subproblems

This means that the problem can be broken down into smaller subproblems, and the same subproblems appear multiple times.

Example: Fibonacci Sequence

The Fibonacci function is defined as:

$$F(n) = F(n-1) + F(n-2), \quad \text{with } F(0) = 0, F(1) = 1$$

If we compute $F(5)$ using naive recursion, the recursive calls look like this:

$$\begin{aligned} F(5) &= F(4) + F(3) \\ &= (F(3) + F(2)) + (F(2) + F(1)) \\ &= \dots \end{aligned}$$

Notice that $F(2)$ and $F(3)$ are calculated multiple times. This is the key idea behind overlapping subproblems.

Why It Matters

Instead of recomputing the same values over and over, dynamic programming stores the results of already-solved subproblems (using memoization or a table) and reuses them. This dramatically improves performance.

2. Optimal Substructure

This means that the optimal solution of the problem can be constructed from the optimal solutions of its subproblems.

Example: Shortest Path in a Graph (Dijkstra's Algorithm)

Suppose we want to find the shortest path from node A to node D in a graph. Let's say the shortest path from A to D goes through B , and the shortest path from B to D goes through C .

Then:

$$\text{ShortestPath}(A \rightarrow D) = \text{ShortestPath}(A \rightarrow B) + \text{ShortestPath}(B \rightarrow C) + \text{ShortestPath}(C \rightarrow D)$$

If each of these segments is already the shortest, then their combination is also the shortest path. This is the idea of optimal substructure.

Why It Matters

If the sub-solutions weren't optimal, then combining them wouldn't guarantee a globally optimal result. Dynamic programming works because we can safely build optimal answers from optimal parts.

In short,

- **Overlapping Subproblems:** Avoids redundant computations by reusing stored results.
- **Optimal Substructure:** Allows building the global solution from local optima.

If both of these properties hold, dynamic programming is usually a good fit for solving the problem efficiently.

DP is widely used in algorithm design and is an essential tool in competitive programming and computer science.

Why Not Plain Recursion?

Recursive solutions often recompute the same values multiple times. For example, the naive recursive Fibonacci function has exponential time complexity.

DP solves this by storing (or “memoizing”) the results of subproblems so they are computed only once.

Steps to Solve a DP Problem

1. Characterize the structure of an optimal solution.
2. Define the recurrence relation.
3. Implement the solution using either:
 - Top-Down (Memoization): recursion + cache
 - Bottom-Up (Tabulation): iterative solution

Example: Fibonacci Numbers

The Fibonacci sequence is defined as:

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2)$$

1. Naive Recursive Solution (Inefficient)

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n-1) + fib(n-2)
```

Time complexity: $O(2^n)$

2. Top-Down DP (Memoization)

Memoization is a technique used in dynamic programming (DP) where the results of expensive function calls are stored (cached) and reused when the same inputs occur again.

It is a **top-down** approach: you write a recursive solution and add a memo (typically a dictionary or array) to store intermediate results.

Memoization prevents the recomputation of the same subproblem multiple times, turning exponential-time recursive solutions into efficient linear-time solutions.

Fibonacci Example with Memoization

We consider the Fibonacci function:

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{otherwise} \end{cases}$$

Python code with memoization:

```
def fib(n, memo={}):  
    if n <= 1:  
        return n  
    if n in memo:  
        return memo[n]  
    memo[n] = fib(n-1, memo) + fib(n-2, memo)  
    return memo[n]
```

Step-by-Step Simulation of fib(5)

We simulate the execution of `fib(5)` step by step.

1. Call `fib(5)` → not in memo → compute `fib(4) + fib(3)`
2. Call `fib(4)` → compute `fib(3) + fib(2)`
3. Call `fib(3)` → compute `fib(2) + fib(1)`
4. Call `fib(2)` → compute `fib(1) + fib(0)` → returns 1 → store `memo[2] = 1`
5. Call `fib(1)` → base case → return 1
6. `fib(3) = fib(2) + fib(1) = 1 + 1 = 2` → store `memo[3] = 2`
7. Call `fib(2)` → found in memo → return 1
8. `fib(4) = fib(3) + fib(2) = 2 + 1 = 3` → store `memo[4] = 3`
9. Call `fib(3)` → found in memo → return 2
10. `fib(5) = fib(4) + fib(3) = 3 + 2 = 5` → store `memo[5] = 5`

Final result: `fib(5) = 5`

Final memo table:

`memo = {2: 1, 3: 2, 4: 3, 5: 5}`

Why is Memoization Linear Time?

Naive Recursion is Exponential

Without memoization, the recursive Fibonacci function recomputes the same values repeatedly. For example, `fib(3)` and `fib(2)` are called multiple times, leading to:

$$T(n) = T(n-1) + T(n-2) + O(1) \Rightarrow T(n) = \Theta(2^n)$$

This creates a recursion tree of exponential size.

Memoization is Linear

In memoization:

- Each unique subproblem `fib(k)` for $k = 0, 1, 2, \dots, n$ is computed **only once**.
- After that, every recursive call is just a dictionary lookup.
- So there are at most n function calls and each takes $O(1)$ after memoization.

Therefore:

$$T(n) = \Theta(n)$$

Key Takeaways

- Memoization avoids recomputation by storing subproblem results.
- Converts exponential-time recursion into linear time.
- Only one computation per subproblem $\rightarrow O(n)$ total time for Fibonacci.
- Slight overhead of recursion remains, but it is manageable for moderate n .

3. Bottom-Up DP (Tabulation)

Tabulation is a bottom-up approach in dynamic programming where we solve all subproblems first and build up the solution to the original problem iteratively.

- We fill a table (usually an array) from the smallest subproblem to the largest.
- No recursion is used.
- The results of smaller subproblems are used to compute larger ones.

Difference from Memoization:

- Memoization is top-down and uses recursion with caching.
- Tabulation is bottom-up and uses iteration without recursion.

Fibonacci Example using Tabulation

We compute the Fibonacci number at position n using an array to store all intermediate values.

Python Implementation

```
def fib_tab(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

Step-by-Step Simulation for fib_tab(5)

We create an array `dp` of size $n + 1 = 6$, initialized as:

`dp = [0, 1, 0, 0, 0, 0]`

We now iterate and fill up the array from index 2 to 5:

- $i = 2$: `dp[2] = dp[1] + dp[0] = 1 + 0 = 1`
- $i = 3$: `dp[3] = dp[2] + dp[1] = 1 + 1 = 2`
- $i = 4$: `dp[4] = dp[3] + dp[2] = 2 + 1 = 3`
- $i = 5$: `dp[5] = dp[4] + dp[3] = 3 + 2 = 5`

Final `dp` table:

`dp = [0, 1, 1, 2, 3, 5]`

Result: `fib_tab(5)` returns 5

Runtime and Space Complexity

Time Complexity

- The loop runs from 2 to n , performing constant work per iteration.
- $\Rightarrow \boxed{O(n)}$

Space Complexity

- We use an array of size $n + 1$ to store intermediate results.
- $\Rightarrow \boxed{O(n)}$

Optimization Tip

If we only need the final Fibonacci number and not the full array, we can reduce space usage to $O(1)$ by keeping only the last two values.

```
def fib_optimized(n):  
    if n <= 1:  
        return n  
    a, b = 0, 1  
    for _ in range(2, n + 1):  
        a, b = b, a + b  
    return b
```

This version uses only two variables and also runs in $O(n)$ time with $O(1)$ space. In short,

- Tabulation builds the solution bottom-up using iteration.
- All subproblems are solved, and the final answer is derived from the full table.
- It avoids recursion, so it may be more space-efficient and avoids stack overflow.
- Time and space complexity are both $O(n)$ in the Fibonacci example.

Time complexity: $O(n)$

Space complexity: $O(n)$